

实验一

练习一：

使用 `test` 指令来判断循环条件：

```
dec %rdi
mov %dl, (%rdi)           // 结果写内存
test %rax, %rax           // 判断是否结束，TODO：请添加判断结束的指令
jnz convert_loop
```

交换 `mov %rdi, %rsi` 和 `mov $1, %rdi` 这两句代码即可：

```
// 系统调用号 (sys_write)      // TODO: 请修复此处的系统调用以正常输出字符串
mov $1, %rax
// 指向字符串的指针
mov %rdi, %rsi
// 文件描述符 (stdout)
mov $1, %rdi
// 要写入的字节数
mov %rdx, %rdx
// 执行系统调用
syscall
```

编译后输出正确：

```
• (base) jzd@DESKTOP-75609C8:~/lab1$ mkdir -p build && cd build
• (base) jzd@DESKTOP-75609C8:~/lab1/build$ cmake -B . -S ../ && cmake --build ./ --target lab1_print_integer
-- CMAKE_BUILD_TYPE is Debug
-- Configuring done (0.6s)
-- Generating done (0.0s)
-- Build files have been written to: /home/jzd/lab1/build
[ 50%] Building ASM object src/lab1/CMakeFiles/lab1_print_integer.dir/print_integer.S.o
[100%] Linking ASM executable ../dist/bins/lab1_print_integer
[100%] Built target lab1_print_integer
• (base) jzd@DESKTOP-75609C8:~/lab1/build$ cd dist/bins/ && ./lab1_print_integer
1234567890123456789
```

练习二：

直接加载矩阵 B：

```
// TODO: 将矩阵B的地址存入MAT_B宏对应的寄存器
mov %rdx, MAT_B
```

计算索引，加载 `A[m][k]`：

先将 `m` 赋值到变量中，在加上 `m` 与 `K` 的乘积，再加上当前的 `k` 获得该数组元素的索引。

```
// TODO: 加载A[m][k]
mov loop_m, mat_elem_idx
imul DIM_K, mat_elem_idx
add loop_k, mat_elem_idx
```

同样，计算索引，加载 `B[k][n]` 和 `C[m][n]`

```
// TODO: 加载B[k][n]
mov loop_k, mat_elem_idx
imul DIM_N, mat_elem_idx
add loop_n, mat_elem_idx

// TODO: 加载C[m][n]
mov loop_m, mat_elem_idx
imul DIM_N, mat_elem_idx
add loop_n, mat_elem_idx
```

能够通过测试:

```
-- Build files have been written to: /home/jzd/lab1/build
[ 14%] Building CXX object _deps/googletest-build/googletest/CMakeFiles/gtest.dir/src/gtest-all.cc.o
[ 28%] Linking CXX static library ../../lib/libgtest.a
[ 28%] Built target gtest
[ 42%] Building CXX object _deps/googletest-build/googletest/CMakeFiles/gtest_main.dir/src/gtest_main.cc.o
[ 57%] Linking CXX static library ../../lib/libgtest_main.a
[ 57%] Built target gtest_main
[ 71%] Building CXX object src/lab1/CMakeFiles/lab1_test_gemm_kernel.unittest.dir/test_gemm_kernel.cpp.o
[ 85%] Building ASM object src/lab1/CMakeFiles/lab1_test_gemm_kernel.unittest.dir/gemm_kernel.S.o
/home/jzd/lab1/src/lab1/gemm_kernel.S: Assembler messages:
/home/jzd/lab1/src/lab1/gemm_kernel.S:95: Warning: translating to `faddp %st,%st(1)'
/home/jzd/lab1/src/lab1/gemm_kernel.S:115: Info: macro invoked from here
[100%] Linking CXX executable ../../dist/bins/lab1_test_gemm_kernel.unittest
/usr/bin/ld: warning: CMakeFiles/lab1_test_gemm_kernel.unittest.dir/gemm_kernel.S.o: missing .note.GNU-stack section implies execut
/usr/bin/ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
[100%] Built target lab1_test_gemm_kernel.unittest
(base) jzd@DESKTOP-75609C8:~/lab1/build$ ./dist/bins/lab1_test_gemm_kernel.unittest --gtest_filter=gemm_kernel.test0
Running main() from /home/jzd/lab1/build/_deps/googletest-src/googletest/src/gtest_main.cc
Note: Google Test filter = gemm_kernel.test0
[=====] Running 1 test from 1 test suite.
[-----] Global test environment set-up.
[-----] 1 test from gemm_kernel
[ RUN    ] gemm_kernel.test0
[       OK ] gemm_kernel.test0 (74 ms)
[-----] 1 test from gemm_kernel (74 ms total)

[-----] Global test environment tear-down
[=====] 1 test from 1 test suite ran. (74 ms total)
[ PASSED ] 1 test.
```

编译上层代码得到输出:

```
• (base) jzd@DESKTOP-75609C8:~/lab1/build$
./dist/bins/lab1_gemm 256 256 256
GEMM performance info:
      M, K, N: 256, 256, 256
      Ops: 0.0335544
      Total compute time(s): 2.3205
      Cost(s): 0.0116025
      Benchmark(Gflops): 2.89199
```

练习四:

查看 lab1_gemm 程序的缓存使用情况:

```
(nav) lenovo@mpi2:~/lab1/build$ sudo perf stat ./dist/bins/lab1_gemm 256 256 256
[sudo] lenovo 的密码:
对不起，请重试。
[sudo] lenovo 的密码:
GEMM performance info:
      M, K, N: 256, 256, 256
      Ops: 0.0335544
      Total compute time(s): 2.53918
      Cost(s): 0.0126959
      Benchmark(Gflops): 2.64294

Performance counter stats for './dist/bins/lab1_gemm 256 256 256':

      2,674.18 msec task-clock                #    1.000 CPUs utilized
           7      context-switches           #    2.618 /sec
           0      cpu-migrations             #    0.000 /sec
          324      page-faults               #   121.159 /sec
  11,869,150,490      cycles                 #    4.438 GHz
  49,468,640,920      instructions           #    4.17 insn per cycle
  3,539,989,587      branches               #    1.324 G/sec
   13,870,432      branch-misses            #    0.39% of all branches

      2.674576465 seconds time elapsed

      2.674500000 seconds user
      0.000000000 seconds sys
```

查看指定的性能事件：

```
(nav) lenovo@mpi2:~/lab1/build/dist/bins$ sudo perf stat -e L1-dcache-loads,L1-dcache-load-misses,dTLB-loads,dTLB-load-misses ./lab1_gemm 256 256 256
GEMM performance info:
  M, K, N: 256, 256, 256
  Ops: 0.0335544
  Total compute time(s): 2.54174
  Cost(s): 0.0127087
  Benchmark(Gflops): 2.64027

Performance counter stats for './lab1_gemm 256 256 256':

   7,065,946,825      L1-dcache-loads
   232,247,880       L1-dcache-load-misses   #    3.29% of all L1-dcache accesses
   7,065,946,825      dTLB-loads
         3,590       dTLB-load-misses       #    0.00% of all dTLB cache accesses

   2.676720373 seconds time elapsed

   2.676631000 seconds user
   0.000000000 seconds sys
```

实验二

练习一：

查看基准代码 lab2_gemm_baseline 的性能分析：

```
(nav) lenovo@mpi2:~/lab2/build/dist/bins$ sudo perf stat -e l2_rqsts.code_rd_hit,l2_rqsts.references,l1d.replacement,l1d_pend_miss.pending,l2_rqsts.pf_hit,l2_rqsts.pf_miss,L1-dcache-loads,L1-dcache-load-misses ./lab2_gemm_baseline 256 1024 256
[sudo] lenovo 的密码:
GEMM performance info:
  M, K, N: 256, 1024, 256
  Ops: 0.134218
  Total compute time(s): 14.1415
  Cost(s): 0.0707077
  Benchmark(Gflops): 1.8982

Performance counter stats for './lab2_gemm_baseline 256 1024 256':

   364,592      l2_rqsts.code_rd_hit              (49.98%)
  1,840,302,892 l2_rqsts.references                (49.99%)
   928,369,198 l1d.replacement                    (50.02%)
  2,694,403,475 l1d_pend_miss.pending              (50.02%)
  1,078,622,835 l2_rqsts.pf_hit                    (50.02%)
   677,613,808 l2_rqsts.pf_miss                    (50.01%)
 28,277,915,678 L1-dcache-loads                    (49.98%)
   930,182,187 L1-dcache-load-misses   #    3.29% of all L1-dcache accesses (49.98%)

 14.842240964 seconds time elapsed

 14.837562000 seconds user
   0.004001000 seconds sys
```

练习二：

使用 imbench 查看各级缓存的访问延迟：

```
(nav) Lenovo@mpi2:~/tools/lmbench-3.0-a9$ cd results && make LIST=$(../scripts/os)/*
x86_64-linux-gnu: using 0.524288 size for memory write timing

L M B E N C H 3 . 0   S U M M A R Y
-----
(Alpha software, do not distribute)

Basic system parameters
-----
Host      OS Description      Mhz  tlb  cache  mem  scal
          pages line  par  load
          bytes
-----
mpi2      Linux 5.15.0-      x86_64-linux-gnu 2280                                1

Processor, Processes - times in microseconds - smaller is better
-----
Host      OS  Mhz null null      open slct sig  sig  fork exec sh
          call I/O stat clos TCP  inst hndl proc proc proc
-----
mpi2      Linux 5.15.0- 2280

Basic integer operations - times in nanoseconds - smaller is better
-----
Host      OS  intgr intgr  intgr  intgr  intgr
          bit  add   mul   div   mod
-----
mpi2      Linux 5.15.0-

Basic uint64 operations - times in nanoseconds - smaller is better
-----
Host      OS  int64 int64  int64  int64  int64
          bit  add   mul   div   mod
-----
mpi2      Linux 5.15.0-

Basic float operations - times in nanoseconds - smaller is better
-----
Host      OS  float float float float
          add mul  div  bogo
-----
mpi2      Linux 5.15.0-

Basic double operations - times in nanoseconds - smaller is better
-----
Host      OS  double double double double
          add  mul  div  bogo
-----
mpi2      Linux 5.15.0-

Context switching - times in microseconds - smaller is better
-----
Host      OS  2p/0K 2p/16K 2p/64K 8p/16K 8p/64K 16p/16K 16p/64K
          ctxsw ctxsw ctxsw ctxsw ctxsw ctxsw ctxsw
-----
```

添加预取，提前将下面要用到的数据取到 cache 中，能通过功能测试：

```

(base) jzd@DESKTOP-75609C8:~/lab2/build$ cd dist/bins && ./lab2_gemm_kernel_opt_prefetch.unittest
Running main() from /home/jzd/lab2/build/_deps/googletest-src/googletest/src/gtest_main.cc
[=====] Running 4 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 4 tests from gemm_kernel_opt_prefetch
[ RUN      ] gemm_kernel_opt_prefetch.test0
[ OK       ] gemm_kernel_opt_prefetch.test0 (16 ms)
[ RUN      ] gemm_kernel_opt_prefetch.test1
[ OK       ] gemm_kernel_opt_prefetch.test1 (4 ms)
[ RUN      ] gemm_kernel_opt_prefetch.test2
[ OK       ] gemm_kernel_opt_prefetch.test2 (5 ms)
[ RUN      ] gemm_kernel_opt_prefetch.test3
[ OK       ] gemm_kernel_opt_prefetch.test3 (1 ms)
[-----] 4 tests from gemm_kernel_opt_prefetch (27 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (27 ms total)
[ PASSED  ] 4 tests.

```

查看性能提升：

```

(base) jzd@DESKTOP-75609C8:~/lab2/build$ ./dist/bins/lab2_gemm_opt_prefetch 1024 128 4
--- Performance before prefetch optimization ---
GEMM performance info:
      M, K, N: 1024, 128, 4
      Ops: 0.00104858
      Total compute time(s): 0.095977
      Cost(s): 0.000479885
      Benchmark(Gflops): 2.18506
--- Performance for after prefetch optimization ---
GEMM performance info:
      M, K, N: 1024, 128, 4
      Ops: 0.00104858
      Total compute time(s): 0.129884
      Cost(s): 0.00064942
      Benchmark(Gflops): 1.61463
-----
Performance difference(Gflops): -0.570422_

```

实验三

练习一：
计算乘积：

```

// TODO 练习1: 添加计算A[m][k] * B[k][n+1] --> st(0)的逻辑
add $1, mat_elem_idx           // n+1
flds (MAT_B, mat_elem_idx, 4)   // 加载 B[k][n+1]
fmul %st(2), %st(0)             // A[m][k] * B[k][n+1]

```

加载和计算：

```

// TODO 练习1: 请添加加载C[m][n] --> st(1) 和 C[m][n+1] --> st(0)的逻辑

// TODO 练习1: 请添加部分和累加逻辑: C[m][n+1] + A[m][k] * B[k][n+1] 和 C[m][n] + A[m][k] * B[k][n]
flds (MAT_C, mat_elem_idx, 4)   // 加载 C[m][n]
faddp %st(2), %st(0)             // C[m][n] + A[m][k] * B[k][n]

add $1, mat_elem_idx           // n+1
flds (MAT_C, mat_elem_idx, 4)   // 加载 C[m][n+1]
faddp %st(1), %st(0)            // C[m][n+1] + A[m][k] * B[k][n+1]

fstps (MAT_C, mat_elem_idx, 4)   // 保存 C[m][n+1]

```

保存结果并更新循环逻辑：

```

// TODO 练习1: 请添加保存C[m][n]的逻辑
sub $1, mat_elem_idx
fstps (MAT_C, mat_elem_idx, 4)   // 保存 C[m][n]

// TODO 练习1: 请添加N维度的循环更新逻辑
add $2, loop_n                   // 每次处理两个n

```

能够通过功能测试：

```

(base) jzd@DESKTOP-75609C8:~/lab3/build$ ./dist/bins/lab3_gemm_opt_loop_unrolling.unittest
Running main() from /home/jzd/lab3/build/_deps/googletest-src/googletest/src/gtest_main.cc
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from gemm_kernel_opt_loop_unrolling
[ RUN    ] gemm_kernel_opt_loop_unrolling.test0
[ OK     ] gemm_kernel_opt_loop_unrolling.test0 (1 ms)
[ RUN    ] gemm_kernel_opt_loop_unrolling.test1
[ OK     ] gemm_kernel_opt_loop_unrolling.test1 (1 ms)
[ RUN    ] gemm_kernel_opt_loop_unrolling.test2
[ OK     ] gemm_kernel_opt_loop_unrolling.test2 (1 ms)
[-----] 3 tests from gemm_kernel_opt_loop_unrolling (4 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (4 ms total)
[ PASSED ] 3 tests.

```

分析性能提升:

```

(base) jzd@DESKTOP-75609C8:~/lab3/build$ ./dist/bins/lab3_gemm_opt_loop_unrolling 256 256 256
--- Performance before loop unrolling optimization ---
GEMM performance info:
      M, K, N: 256, 256, 256
      Ops: 0.0335544
      Total compute time(s): 2.95053
      Cost(s): 0.0147526
      Benchmark(Gflops): 2.27447
--- Performance for after loop unrolling optimization ---
GEMM performance info:
      M, K, N: 256, 256, 256
      Ops: 0.0335544
      Total compute time(s): 2.43734
      Cost(s): 0.0121867
      Benchmark(Gflops): 2.75336
-----
Performance difference(Gflops): 0.478891

```

练习二:

加载矩阵 C 的部分至 AVX 寄存器:

```

// TODO 练习3: 请添加加载C[m][n:n+32]-->mat_c0_0_8, mat_c0_8_16, mat_c0_16_24, mat_c0_24_32的逻辑
vmovups (MAT_C, mat_elem_idx), mat_c0_0_8
vmovups 32(MAT_C, mat_elem_idx), mat_c0_8_16
vmovups 64(MAT_C, mat_elem_idx), mat_c0_16_24
vmovups 96(MAT_C, mat_elem_idx), mat_c0_24_32

```

```

// 装载矩阵C第二行的数据, 即C[m+1][n:n+32]
mov temp_reg, mat_elem_idx
add DIM_N, mat_elem_idx
shl $2, mat_elem_idx // 左移, 相当于乘4

```

```

// TODO 练习3: 请添加加载C[m+1][n:n+32]-->mat_c1_0_8, mat_c1_8_16, mat_c1_16_24, mat_c1_24_32的逻辑
vmovups (MAT_C, mat_elem_idx), mat_c1_0_8
vmovups 32(MAT_C, mat_elem_idx), mat_c1_8_16
vmovups 64(MAT_C, mat_elem_idx), mat_c1_16_24
vmovups 96(MAT_C, mat_elem_idx), mat_c1_24_32

```

加载并广播 A 的逻辑:

```

// TODO 练习3: 请添加加载并广播A[m+1][k]-->mat_a1_0_8的逻辑
mov loop_m, %rax          // rax = m
add $1, %rax              // rax = m+1
mul DIM_K                 // rax = (m+1)*K
mov %rax, temp_reg        // temp_reg = (m+1)*K
add loop_k, temp_reg       // temp_reg = (m+1)*K+k

mov temp_reg, mat_elem_idx // mat_elem_idx = (m+1)*K+k
shl $2, mat_elem_idx       // mat_elem_idx = 4*(m+1)*K+4*k

vbroadcastss (MAT_A, mat_elem_idx), mat_a1_0_8 // 将A[m+1][k]广播到AVX寄存器的8个单元

```

装载矩阵 B:

```

.macro LOAD_MAT_B      // 每次装载矩阵B一行32个元素，即B[k][n:n+32]

    // TODO 练习3: 请添加加载B[k][n:n+32]-->mat_b0_0_8, mat_b0_8_16, mat_b0_16_24, mat_b0_24_32的逻辑
    mov loop_k, %rax          // rax = k
    mul DIM_N                 // rax = k * N
    mov %rax, temp_reg        // temp_reg = k * N
    add loop_n, temp_reg       // temp_reg = k * N + n

    mov temp_reg, mat_elem_idx // mat_elem_idx = k * N + n
    shl $2, mat_elem_idx       // mat_elem_idx = 4 * (k * N + n)

    vmovups (MAT_B, mat_elem_idx), mat_b0_0_8 // 将B[k][n:n+8]装载到mat_b0_0_8
    vmovups 32(MAT_B, mat_elem_idx), mat_b0_8_16 // 将B[k][n+8:n+16]装载到mat_b0_8_16
    vmovups 64(MAT_B, mat_elem_idx), mat_b0_16_24 // 将B[k][n+16:n+24]装载到mat_b0_16_24
    vmovups 96(MAT_B, mat_elem_idx), mat_b0_24_32 // 将B[k][n+24:n+32]装载到mat_b0_24_32

.endm

```

计算对应的矩阵 C:

```

.macro DO_COMPUTE      // 计算 C[m:m+2][n:n+32] += A[m:m+2][k] * B[k:k+8][n:n+32]

    // TODO 练习3: 请添加计算C[m:m+2][n:n+32] += A[m:m+2][k] * B[k:k+8][n:n+32]的逻辑
    // 计算第一行: C[m][n:n+32] += A[m][k] * B[k][n:n+32]
    vfmadd231ps mat_b0_0_8, mat_a0_0_8, mat_c0_0_8
    vfmadd231ps mat_b0_8_16, mat_a0_0_8, mat_c0_8_16
    vfmadd231ps mat_b0_16_24, mat_a0_0_8, mat_c0_16_24
    vfmadd231ps mat_b0_24_32, mat_a0_0_8, mat_c0_24_32

    // 计算第二行: C[m+1][n:n+32] += A[m+1][k] * B[k][n:n+32]
    vfmadd231ps mat_b0_0_8, mat_a1_0_8, mat_c1_0_8
    vfmadd231ps mat_b0_8_16, mat_a1_0_8, mat_c1_8_16
    vfmadd231ps mat_b0_16_24, mat_a1_0_8, mat_c1_16_24
    vfmadd231ps mat_b0_24_32, mat_a1_0_8, mat_c1_24_32

.endm

```

保存矩阵 C:

```

// TODO 练习3: 请添加保存mat_c0_0_8, mat_c0_8_16, mat_c0_16_24, mat_c0_24_32 --> C[m][n:n+32]的逻辑
vmovups mat_c0_0_8, (MAT_C, mat_elem_idx)
vmovups mat_c0_8_16, 32(MAT_C, mat_elem_idx)
vmovups mat_c0_16_24, 64(MAT_C, mat_elem_idx)
vmovups mat_c0_24_32, 96(MAT_C, mat_elem_idx)

```

能够通过功能测试:

```

(base) jzd@DESKTOP-75609C8:~/lab3/build$ ./dist/bins/lab3_gemm_opt_avx.unittest
Running main() from /home/jzd/lab3/build/_deps/googletest-src/googletest/src/gtest_main.cc
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from gemm_kernel_opt_avx
[ RUN    ] gemm_kernel_opt_avx.test0
[ OK     ] gemm_kernel_opt_avx.test0 (0 ms)
[ RUN    ] gemm_kernel_opt_avx.test1
[ OK     ] gemm_kernel_opt_avx.test1 (0 ms)
[ RUN    ] gemm_kernel_opt_avx.test2
[ OK     ] gemm_kernel_opt_avx.test2 (1 ms)
[-----] 3 tests from gemm_kernel_opt_avx (2 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (2 ms total)
[ PASSED ] 3 tests.

```

查看性能分析：

```

(base) jzd@DESKTOP-75609C8:~/lab3/build$ ./dist/bins/lab3_gemm_opt_avx 256 256 256
--- Performance before avx optimization ---
GEMM performance info:
      M, K, N: 256, 256, 256
      Ops: 0.0335544
      Total compute time(s): 2.95115
      Cost(s): 0.0147558
      Benchmark(Gflops): 2.27399
--- Performance for after avx optimization ---
GEMM performance info:
      M, K, N: 256, 256, 256
      Ops: 0.0335544
      Total compute time(s): 0.119478
      Cost(s): 0.00059739
      Benchmark(Gflops): 56.1684
-----
Performance difference(Gflops): 53.8944  _

```